



Event modding

Events are one of the most elementary forms of content and are simultaneously easy to learn. They present the nation with a choice or serve as notifications for something occurring.

Please help with verifying or updating older sections of this article.
At least some were last verified for version pre-release.

İçindekiler

File location

Structure

- Namespace
- Inline scripted triggers/effects
- Modifying base game events

Basic event structure

- Event types and categories
- Title, description and historical info
- Immediate and after
- Trigger
- Fire only once and orphan
- Dynamic historical event
- Major events
- Hidden
- Interface lock
- Weight multiplier
- Outcome

Option Structure

- Dynamic name string
- AI selection modding
- Historical option
- Unused fields

[Editing an event](#)

References

File location



Please help with verifying or updating this section. It was last verified for version pre-release.

All event files in Europa Universalis V need to be contained in the `events` folder or in one of its subfolders. For example:

- `/Europa Universalis V/game/in_game/events/innovativeness_events.txt`
- `/Europa Universalis V/game/in_game/events/DHE/flavor_plc.txt`

Structure



Please help with verifying or updating this section. It was last verified for version pre-release.

The event file usually consists of:

- One or more namespaces corresponding to the event ids in the file
- Inline scripted triggers / effects
- The events themselves

```
namespace = test_events
scripted_trigger test_trigger = {
    ...
}

scripted_effect test_effect = {
    ...
}

test_events.1 = {
    type = country_event
    title = test_events.1.title
    desc = test_events.1.desc
    ...
}
```

Namespace

Every event name needs to be formatted as `namespace.integer` where the integer is > 0 and < 10000 . Not adding the namespace will lead to situations where events may overlap and not fire as intended.

Inline scripted triggers/effects

Event files exclusively allow the creation of **inline scripted triggers/effects** which work like the regular scripted effects and triggers but can be used only in the file they were created in. Their definition must be preceded by `scripted_effect` / `scripted_trigger` accordingly.



Modifying base game events

See also: [*Mod compatibility*](#)

Direct file overwrites – using a file with the same name and path as base game file – are discouraged, as it creates difficulty for inter-mod compatibility and maintenance for base game updates. This guide describes an alternative approach.

Database entry modes REPLACE: and INJECT: do **not** work for this purpose. However, the method below allows you to modify individual base game events. It produces notification errors in error.log, but not cause any actual errors or unexpected behavior.

1. Create a new event file in the mod's events folder. Its filename must come before the relevant base game files in ascii order, for example by starting with 0000_ like 0000_modded_events.txt.
2. At the top of the new file, add the namespace of the event to be modify. For example:
 - namespace = earthquake_events
3. Copy the specific events in that namespace that will be modified into the new file — for example, just earthquake_events.1.
4. Multiple namespaces can be used in one file as long as all events follow their namespace directly
5. Make any desired edits to the events.
6. If the event uses scripted effects or triggers that are *defined in the base game event file*, copy the *contents* of those scripted effects (not the entire scripted effect block) into the modified events or copy the scripted effect or trigger with a modified name.
 - Copying the scripted effect or trigger without changing its name breaks it for the rest of the base game file; copying nothing breaks it in the new file.
7. The edits are now applied correctly as the new file is loaded before the base game file.
8. Note that a Duplicated event ID error will appear in error.log for each modified event. This is expected and harmless.

Basic event structure



Please help with verifying or updating this section. It was last verified for [version](#) pre-release.

```

test_events.1 = {
    type = country_event                # type of event
    title = test_events.1.title         # title key to be localized
    desc = test_events.1.desc           # description key to be localized
    historical_info = test_events.1.historical_info # optional description text to provide historical
context

    trigger = {                        # List of requirements that need to be fulfilled for event to fire
        is_subject = no
    }

    fire_only_once = yes               # may fire only once during a campaign, regardless of nation

    dynamic_historical_event = {       # Used to make the event fire for the right tags
        tag = FRA                     # tag that this event can fire for, can provide multiple
        from = 1400.1.1               # the earliest date at which this event can fire
        to = 1500.1.1                 # the latest date at which this event can fire
        monthly_chance = 10           # monthly chance to fire the event (10 = 10%)
    }
}

```



```
name = test_events.1.a # localizable key for event option

historical_option = yes # will mark this option as 'historical' and
                        # if historical AI is enabled, AI will always pick this

add_gold = 25

}

option = {
    name = test_events.1.b # localizable key for second event option

    #...
}

}
```

Event types and categories

There are multiple event types that can be added to type property. The type dictates the scope on which an event can be fired on:

Event Type	Scope	Other Information
country_event	country	Regular type for events fired for country
location_event	location	Regular type for events fired for locations
exploration_event	exploration	Events used for on_exploration_monthly_pulse
unit_event	unit	Unused
age_event	country	Used exclusively for ages_of_eu.1 event. Uses special graphics
omens_event	?	???
No type	country	Very likely the type by default if not specified is country_event

Additionally, events also support a category property. The category is used for additional graphics and to pull the relevant situation/disaster/international_organization icon according to gfx/interface/icons/\$path\$/event_namespace\$.dds.

```
test_events.2 = {
    type = country_event
    category = situation_event

    ...
}
```

The following categories are supported:





io_event	international_organizations/
disaster_event	disasters/

Title, description and historical info

All non-hidden events need a localized title and description. Events based on history can also be provided with historical_info, which adds an additional textbox below the regular description.

The strings provided need to be properly localized.

```
test_events.3 = {
  ...
  title = test_events.3.title
  desc = test_events.3.desc
  historical_info = test_events.3.historical_info
  ...
}
```

All three of the localisation keys can be made dynamic based on triggers with the following structure:

```
test_events.3 = {
  ...
  title = {
    first_valid = {
      triggered_desc = {
        desc = test_events.3.title_1
        trigger = {
          # <triggers>
        }
      }
      triggered_desc = {
        desc = test_events.3.title_2
        trigger = {
          # <triggers>
        }
      }
      desc = test_events.3.fallback_title # Fallback, will fire when nothing above does
    }
  }
  ...
}
```

NOTE: The triggered_desc and desc are correct even though it is used in title. This also applies for historical_info.

One can also use random_valid instead of first_valid, leading to a situation where a random title is chosen.

Those statements can also be nested inside of each other:

```
test_events.3 = {
  ...
  title = {
    first_valid = {
      triggered_desc = {
        trigger = {
```



```

desc = test_events.3.title.random.a
desc = test_events.3.title.random.b
desc = test_events.3.title.random.c
}
}
}
}
desc = test_events.3.title_fallback
}
}
}
...
}

```

In addition to the above, the title also accepts a `switch` statement, which works much like other switch statements:

```

test_events.3 = {
  ...
  title = {
    switch = {
      fallback = test_events.3.title.fallback # Fallback in case none of the
below                                         # are present

      trigger = has_local_variable
      success = test_events.3.title.success
      failure = test_events.3.title.failed_the_war
    }
  }
}

```

Immediate and after

The `immediate` field is used to execute effects before/as the event is shown to the player. Scopes saved in the immediate field can be used in event localisation and are considered for event illustration. Effects from immediate will not be shown in any tooltip.

The `after` field denotes effects that are to be executed after an event option is selected. The effects from after are shown in each option after the rest of the effects. It is recommended that after is used for cleaning up any event effects that may persist after the event.

Trigger

The `trigger` field is used to determine if an event can fire. If the trigger is false, the event will not fire, and, if executed by `on_actions`, its weight will not be considered.

`on_trigger_fail` represents effects that are executed when the game tries to execute the event but is unable to do so due to the trigger.

```

test_events.1 = {
  trigger = {
    has_variable = has_won_war_against_france
  }
  on_trigger_fail = {
    trigger_event_non_silently = test_event.2 #Failed to defeat france
  }
}

```



Fire only once and orphan

The `fire_only_once` field can be used to make the event be fired only once per game. This restriction is applied globally, meaning that if that event fires for one country, it may not fire again, even for a different country. Under the hood, when that event is executed, a global variable is set under the name `<event_name>_fire_only_once`.

`orphan = yes` field is used to prevent errors caused by event not being fired from any known source (`on_action`, `trigger_event`, `dynamic_historical_event`). Can be used for debug events fireable only via console.

Dynamic historical event

The `dynamic_historical_event` field is used to fire events for certain tags made available from a certain date to another.

```
test_events.1 = {
  dynamic_historical_event = {
    tag = POL          # May fire for POL tag
    tag = PLC          # May also fire for PLC tag
    from = 1444.1.1    # The game will check if this event can be fired after this date
    to = 1500.1.1      # The game will stop checking after this date.
    monthly_chance = 10 # Monthly chance that this event will fire (10 = 10% chance)
  }
}
```

When an event is fired with `dynamic_historical_event`, an error message will be printed if the event does not also have `fire_only_once = yes`.

Major events

`major = yes` and `major_trigger` fields are used to announce another nation's event to nations that fulfill the `major_trigger` trigger.

ROOT in the `major_trigger` field is the country being given the popup, and `scope:from` is the nation which got the original event.

```
test_events.1 = {
  major = yes
  major_trigger = {
    gold >= 100
    "opinion(scope:from)" > 100    # scope:from is available here, where from is the country that
    fired the original event
  }
}
```

Hidden





Interface lock

By default, events force the game to pause. One can use `interface_lock = no` to disable this behavior.

Weight multiplier

`weight_multiplier` can be used to increase or decrease the chance of an event being fired when listed in a `random_events` list.

```

random_events = {
    100 = 0
    10 = test_events.1      # if the weight multiplier of this event evaluates to two,
                           # it will have a chance of (10*2) / ((10*2) + 10 + 100)

    10 = test_events.2
}

```

`weight_multiplier` uses the *Mean Time to Happen* (MTTH) notation:

```

test_events.1 = {
    ...

    weight_multiplier = {
        base = 10          # base value for our weight multiplier. The result of the modifier math below
                           # will be compared against this value

        modifier = {
            factor = 2
            trigger = {
                legitimacy >= 50
            }
        }
        modifier = {
            add = 5
            trigger = {
                prestige >= 50
            }
        }
    }
}

```

The resulting MTTH math result is compared against the base value provided to give a resulting modifier which modifies the weight of the event in the random list.

In the above scenario, if the country has ≥ 50 legitimacy but < 50 prestige, the result will be $(10*2)/10 = 2$;

If it is ≥ 50 prestige but < 50 legitimacy, it will be $(10+5)/10 = 1.5$;

For both ≥ 50 prestige and ≥ 50 legitimacy $((10*2)+5)/10 = 2.5$;

And for neither just $10/10 = 1$.

Outcome

`outcome` field can be used to add an additional “positive” and “negative” sound effect for the event.



Option Structure



Please help with verifying or updating this section. It was last verified for version pre-release.

```

option = {
    name = test_events.1.a # localizable key for event option

    historical_option = yes # will mark this option as 'historical' and
                           # if historical AI is enabled, AI will always pick this

    trigger = {
        prestige >= 25
    } # requirements needed for this event to show up at all

    add_gold = 25 # Effects to be executed when option is selected
}

```

Only 5 event options are visible at a time and any extra will be shown with a scrollbar.

Dynamic name string

You can also have dynamic text for event option names, with a different system:

```

option = {
    name = {
        text = test_events.1.a.a
        trigger = {
            prestige > 20
        }
    }
    name = {
        text = test_events.1.a.b
        trigger = {
            prestige <= 20
        }
    }
    name = test_events.1.a.c
    name = test_events.1.a.d
    ...
}

```

If multiple names' triggers return true, one will be chosen at random.

AI selection modding

`ai_will_select` and `ai_weight` are both used for AI to determine which option will be picked. If both are available, `ai_will_select` will be used. Both of them provide the AI with a weight of the option which is selected according to weighted randomness:

```

option = {
    name = test_events.1.a
    ai_will_select = { value = 100 }
}

```



```
option = {
    name = test_events.1.c
    ai_will_select = { value = 20 }
}

# Total weight of all options is 100 + 30 + 20 = 150
# The chance of AI picking option a is 100 / 150 = ~67%
# The second option's chance is 30/150 = 20%
# And the third option's chance is 20/150.
```

ai_will_select determines the value by using a scripted value, while ai_weight uses “mean time to happen”-like syntax.

Historical option

You can mark any amount of options with an event as historical using historical_option = yes. Historical options have an additional icon marking them as such and if, Historical AI gamerule is turned on, AI will always pick those options. If multiple are available, they will be weighed against each other.

Unused fields

There are 4 unused fields that have no inherent effect but can be used for GUI scripting. The following categories are supported:

Field	GUI script function
moral_option	EventOption.IsMoral
evil_option	EventOption.IsEvil
high_risk_option	EventOption.IsHighRisk
high_reward_option	EventOption.IsHighReward

Illustrations



Please help with verifying or updating this section. It was last verified for version pre-release.

You can add a certain event picture to an event using the image field:

```
image = "gfx/interface/illustrations/international_organization_types/autocephalous_patriarchate.dds"
```

Otherwise, an automatic event picture will be generated based on the saved scopes and illustration_tags field.

The game will generate picture based on the following factors:

- Two first saved pop scopes determine pops visible in the event picture. Their mood can be modified using the illustration tags (see illustration_tags below)
- Saved location scope will determine the background.



- As a last resort, the capital location will be taken.

Illustration tags

illustration_tags are used to modify the pops in the event picture. The possible illustration tags can be found over at [gfx/illustrations/database/](#) A maximum of two character portraits can be shown in an event picture by saving them as saved scopes. If that is not desired, they can be hidden using `hide_portraits = yes`.

```
test_events.1 = {
    ...
    #hide_portraits = yes          # Can be used to hide character portraits
                                  # otherwise generated from saved scopes

    immediate = {
        ruler = {
            save_scope_as = target_character          # First character portrait
        }
        random_neighboring_country = {
            ruler = {
                save_scope_as = target_character2      # Second character portrait
            }
        }
        random_cabinet_character = {
            save_scope_as = target_character3          # This character will not be shown
                                                        # as there is only space for two.
        }
    }
}
```

Firing an event

 Please help with verifying or updating this section. It was last verified for [version](#) pre-release.

Events in Europa Universalis V can be fired through three main ways:

- [on_actions](#), in the events field.
- Regular [effects](#), using either the `trigger_event_silently` or `trigger_event_non_silently` effects.
- `dynamic_historical_event` clause in the event.

References

Modding		Return to top
Documentation	Defines • Effects • Scopes • Scope links • Triggers Colors • Macros • Mean time to happen • Modifier types • On actions • Script value • Variables GUI script • Localization	
Scripted content	Actions • Disasters • Events • Missions • Modifiers • Scripted gui • Setup • Situations • Customizable localization	
Scripted types	Advances • Art • Buildings • Bureaucracies • Casus belli • Characters • Concepts • Countries • Culture • Diplomacy • Diseases • Estates • Goods • Institutions • International organizations • Laws • Movements • Peace treaties • Pops • Religion • Subject types • Traits • Units • Wargoals	
Map	Map • Map modes • Terrain	
Graphics	3D Models • Interface • Graphical assets • Fonts • Flags	
Audio	Music • Sound	
Other	AI • Console commands • Checksum • Mods • Mod compatibility • Mod structure • Troubleshooting	
Guides	Interface modding guide • Mod translation • Save-game editing • Settlement position modding guide	



Sayfa en son 18.55, 15 Mayıs 2026 tarihinde değiştirildi.

Aksi belirtilmedikçe içeriğin kullanımı Attribution-ShareAlike 3.0 lisansı kapsamında uygundur.

GAMES

Discover

Our Brands

Browse

Play on Paradox technology

ABOUT

News

About us

Careers

Join our playtests

Media contact

COMMUNITY

Paradox Forums

Paradox Wikis

Support

SOCIAL MEDIA

Terms & Policies

Legal Information

EU Online Dispute Resolution

Report Illegal Content

Frequently Asked Questions

Paradox Interactive corporate website

